WEEK 2 STUDY NOTES

Inside the Transformer: Decoder, Positions & Normalization

Notes from lectures L9, L6, L7, L5, L8 and L10, rewritten in plain language with worked examples and diagrams.

#	Lec.	What you will learn	Key takeaway
1	L9	Decoder basics, teacher forcing, masked attention	Feed the true previous word, but hide the future with a mask
2	L6	Zooming into one decoder layer, cross-attention, output layer	Query from decoder, Keys/Values from encoder
3	L7	Why Transformers need positional information	Attention alone cannot tell word order
4	L5	Sinusoidal positional encoding	sin/cos waves give each position a unique, distance-aware code
5	L8	Residual connections and batch normalization	Skip connections + standardizing activations make deep nets trainable
6	L10	Layer normalization, Add & Norm, the full architecture	Normalize across features of one token; put it all together

HOW TO USE THESE NOTES

Each part starts with the **big idea**, then explains step by step with **examples** and **diagrams**, and ends with a **quick check**. The last page is a one-page cheat sheet of formulas and shapes. The running example is the one used in the lectures: translating an English sentence into Tamil (the first Tamil word is *Naan* = "I"). Other words in diagrams are illustrative.

Contents

PART 1 | LECTURE L9 — The Decoder, Teacher Forcing & Masking 3

PART 2 | LECTURE L6 — Zooming into a Decoder Layer 7

PART 3 | LECTURE L7 — Why Transformers Need Positional Information 11

PART 4 | LECTURE L5 — Sinusoidal Positional Encoding 14

PART 5 | LECTURE L8 — Residual Connections & Batch Normalization 18

PART 6 | LECTURE L10 — Layer Normalization & the Full Transformer 20

PART 1 | LECTURE L9

The Decoder, Teacher Forcing & Masking

How the decoder generates text one word at a time, and why we hide the future

1.1 What does the decoder do?

In translation, the **encoder** reads the English sentence and produces a rich vector for every word. The **decoder** then writes the Tamil sentence **one word at a time**. At each step it looks at two things: (a) what the encoder understood and (b) the words it has already written. Then it predicts the next word. This loop is called **autoregressive decoding** (you already met it with RNNs).

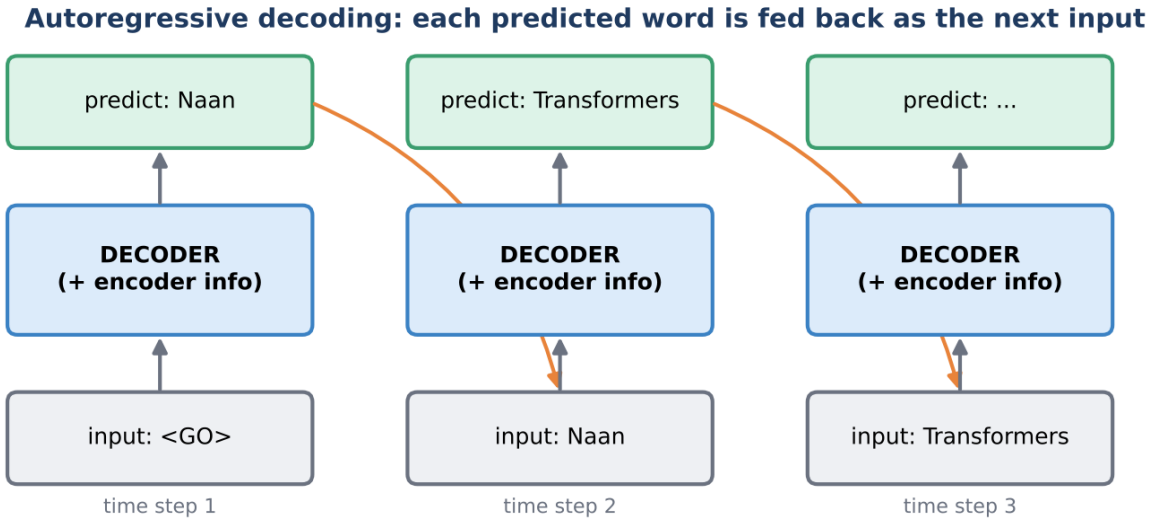


Figure 1. Autoregressive decoding: the predicted word becomes the next input.

EXAMPLE

The decoder gets the encoder's summary of "*I enjoyed the movie Transformers*" and the start symbol **<GO>**. It outputs **Naan** ("I"). Now the input is [<GO>, Naan] and it outputs the next word, and so on until the sentence ends.

Inputs and outputs of the decoder

- **Input 1:** encoder outputs E_1 ... E_T (one vector per English word).
- **Input 2:** embeddings of the words generated so far (starting with <GO>).
- **Output:** a **probability distribution over the whole vocabulary** (a V-dimensional vector, produced by a **softmax**). We want the correct word (*Naan*) to get the highest probability. At test time we simply take the **argmax** (other strategies exist and come later).

The decoder is a **stack of N identical layers** (N = 6 in the original paper; 8, 12, 24 or more in other designs). Every input word has its own vector that travels up through all the layers, exactly like in the encoder. Only the very last layer turns each vector into a probability distribution.

1.2 What is inside one decoder layer?

Encoder layer	Decoder layer
1. Multi-head **self**-attention 2. Feed-forward network	1. **Masked** multi-head self-attention (NEW) 2. Multi-head **cross**-attention (NEW) 3. Feed-forward network

Cross-attention = attention *between* encoder and decoder (the decoder looks at the encoder). That is covered in Part 2. In this part we learn the first new idea: **masked** self-attention. To understand why it exists we need **teacher forcing**.

1.3 Teacher forcing

Imagine we are at the very start of training. All weights are random. We give the decoder <GO> and it predicts some random word, say "Hindi" (any word from the vocabulary).

- **Option A:** feed its own prediction ("Hindi") as the next input. Now it must continue from a wrong word, so its next prediction is junk too, and the errors snowball. Early training would be hopeless.
- **Option B (teacher forcing):** we **know** the right translation from the training data, so feed the **correct** word (*Naan*) as the next input, regardless of what the model predicted.

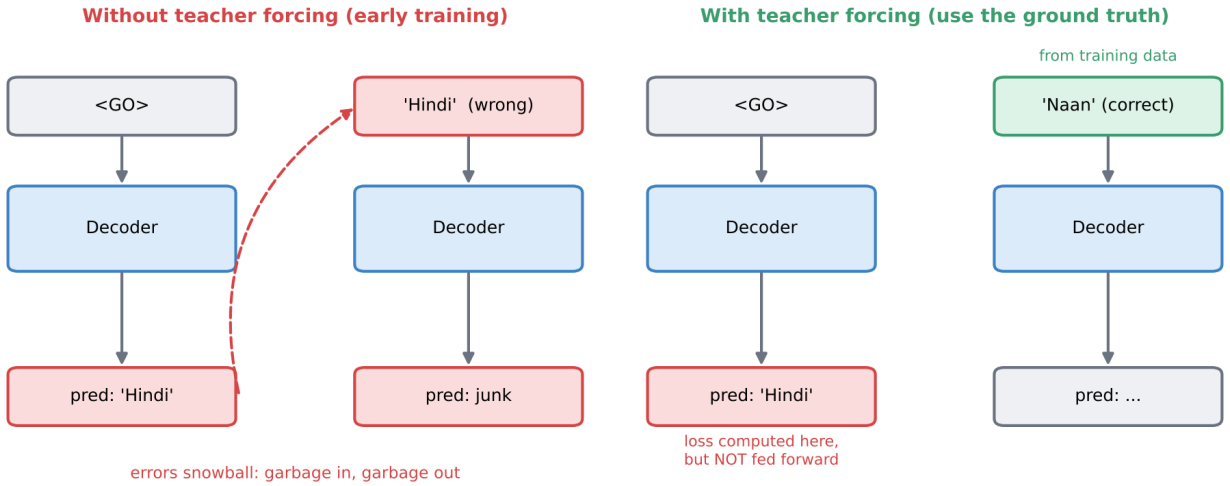


Figure 2. Left: feeding wrong predictions snowballs. Right: teacher forcing feeds the ground truth. The loss is still computed on the (wrong) prediction.

REMEMBER

We **still use the model's prediction to compute the loss** (cross-entropy against the correct word) and back-propagate. We just do not feed that prediction forward as the next input.

But there is a catch

If teacher forcing is allowed to go to the extreme, we would simply feed the **entire** Tamil sentence into the decoder and ask it to predict it. Then the answer is sitting in the input and the model only has to learn the **identity function** (copy the input). It would learn nothing useful, and the encoder would be pointless.

Idea 1: feed the true word	Idea 2: do not leak the answer
Early predictions are random, so do not build on them. Use the ground truth.	At step t the model must **not see words t, t+1, ...** It may only see earlier words. This also matches real life: at test time the future words do not exist yet.

THE BALANCE

Teacher forcing says **"give it the correct previous words"**. Masking says **"but only the ones that come *before* the current position"**. Together: the decoder is trained with the true history, never with the future.

1.4 Masking: what the decoder is allowed to see

If the target sentence is **<GO> Naan w2 w3 ...**, the allowed view at each step is:

Predicting...	Allowed to look at	Hidden (masked)
word 1 (Naan)	<GO>	Naan, w2, w3, ...
word 2 (w2)	<GO>, Naan	w2, w3, ...
word 3 (w3)	<GO>, Naan, w2	w3, ...
word t	everything before t	t and everything after

In the attention score matrix (row = word being processed, column = word it attends to) the allowed cells form a **lower-triangular** pattern: row 1 can use column 1 only, row 2 can use columns 1-2, and so on. The matrix is (T+1) x (T+1), where the +1 is the <GO> token.

1.5 How masking is implemented

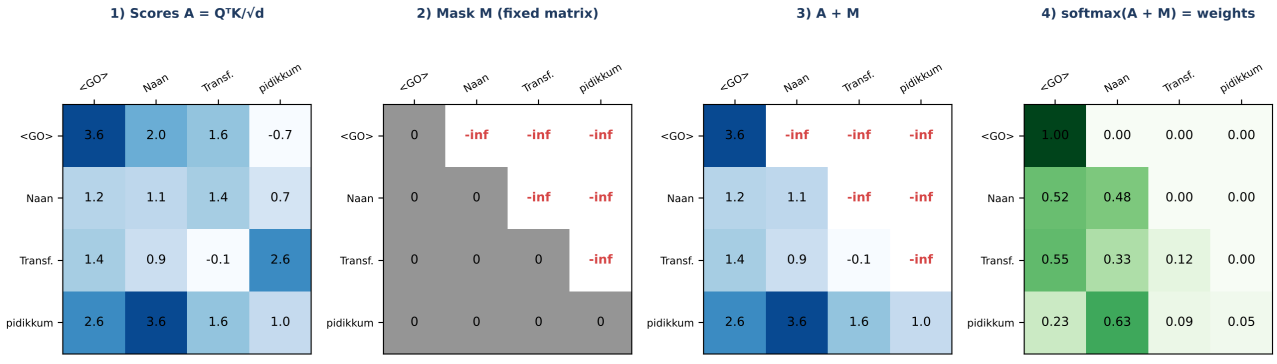
Recall self-attention: scores $\mathbf{A = Q^T K / \sqrt{d}}$, then softmax, then multiply by V. Without a mask, every word attends to every word (a full T x T matrix). We need the softmax output to be **exactly 0** for forbidden cells. The trick is to add a fixed **mask matrix M** to the scores *before* the softmax.

$$\mathbf{Attention = softmax(\ Q^T K / \sqrt{d} + M\ ) \cdot V}$$

- M = **0** on and below the diagonal (allowed cells: scores unchanged).
- M = **-∞** above the diagonal (forbidden cells).

WATCH OUT

Why not add a value that makes the forbidden score *zero*? Because softmax computes e^{score}. If the score is 0 then $e^0 = 1$, not 0, so that word still gets attention. We need $e^{score} = 0$, which only happens when score = **-∞**.



Row = the word being processed, column = the word it looks at. -inf → e^(-inf) = 0 → attention weight exactly 0 (each row still sums to 1).

Figure 3. A 4-word example (illustrative words). The -inf cells become exactly 0 after softmax; every row still sums to 1 over the allowed cells.

WHY THE ROW STILL WORKS

Each row has at least one allowed cell (the word itself and earlier words), so the softmax denominator is never 0. Example row 2: scores 1.2 and 1.1 become weights 0.52 and 0.48.

A nice property: the mask is the **same fixed matrix for every sentence** in the batch (it only depends on the sequence length T), so it is cheap to apply.

1.6 Masking also happens in the encoder (padding mask)

In practice every sentence in a batch is padded to a maximum length.

EXAMPLE

Max length 512, batch size 64. A sentence with only 9 words gets 503 **[PAD]** tokens. PAD tokens carry no meaning, so we mask them (give them -∞ scores) so no real word wastes attention on them. Same mechanism, different purpose.

Quick check	Answer
What is teacher forcing?	Feeding the correct previous word (from the data) as the next decoder input instead of the model's own prediction.
Why not feed the whole target sentence at once?	The answer would leak; the model would just copy and the encoder would be useless. So we mask the future.
Why -∞ and not 0 in the mask?	softmax uses e^x; $e^0 = 1$ still gives attention, $e^{-\infty} = 0$ gives none.
Is the mask different for each sentence?	No. It is one fixed lower-triangular pattern determined by the sequence length.

PART 2 | LECTURE L6

Zooming into a Decoder Layer

Masked self-attention, cross-attention, feed-forward and the output layer

BIG IDEA

A decoder layer refines each word's vector three times: (1) look at earlier decoder words (masked self-attention), (2) look at the encoder's understanding of the input sentence (cross-attention), (3) process each vector on its own (feed-forward). Shapes never change: **T x d in, T x d out**.

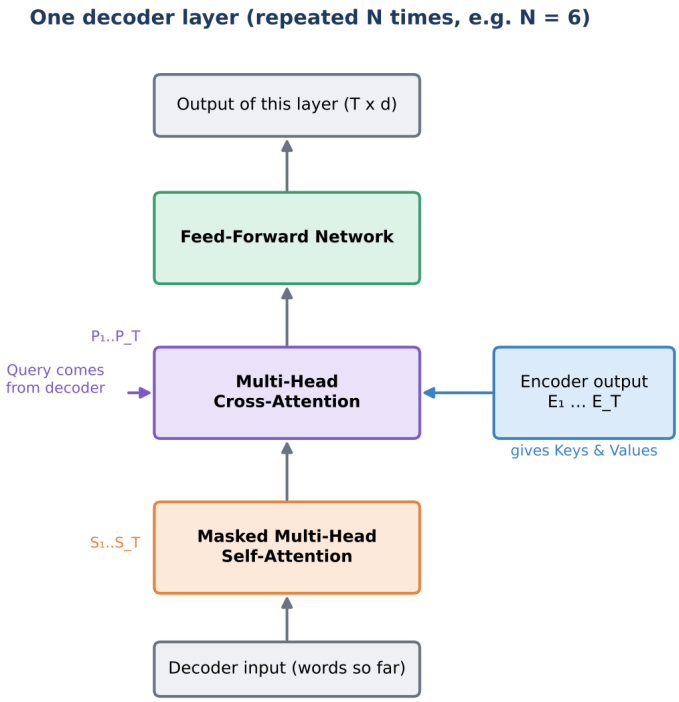


Figure 4. One decoder layer. The encoder output enters only at the cross-attention block.

2.1 Step 1: masked self-attention

During training we have the full target sentence (<GO> Naan w2 ...). For every word's embedding h we compute three vectors using three weight matrices:

$$\mathbf{q = h \cdot W_Q,\ k = h \cdot W_K,\ v = h \cdot W_V}$$

Then we apply the masked attention from Part 1, $\mathbf{softmax(Q^T K/\sqrt{d} + M)V}$, followed by the usual output linear layer. The result is a new vector for each word, called $\mathbf{S_1 \ldots S_T}$. Each S_i has only used information it is allowed to use (words up to i), so it is "legitimate".

REMEMBER

Notation: H = decoder inputs, S = after masked self-attention, P = after cross-attention. The lecturer assumes encoder and decoder lengths are both T for simplicity; in reality they can differ (T_{dec} vs T_{enc}).

2.2 Step 2: cross-attention (decoder looks at encoder)

Now we want S to **interact with the encoder output E**, so each decoder word becomes aware of the English sentence. The output is a new set of vectors $\mathbf{P_1 \ldots P_T}$.

THE ONE RULE TO MEMORIZE

The one who wants a new representation gives the QUERY. The ones being looked at give the KEYS and VALUES.
Here the decoder wants information, so **Q comes from S (decoder)**. The encoder is being consulted, so **K and V come from E (encoder)**.

EXAMPLE

Think of a student (decoder word) in a library (encoder). The student's **question** is the Query. Each book has a **label** (Key) used to check how relevant it is, and **contents** (Value) that you read. The answer is a relevance-weighted mix of the book contents. In self-attention the student and the library are the same people; in cross-attention they are different.

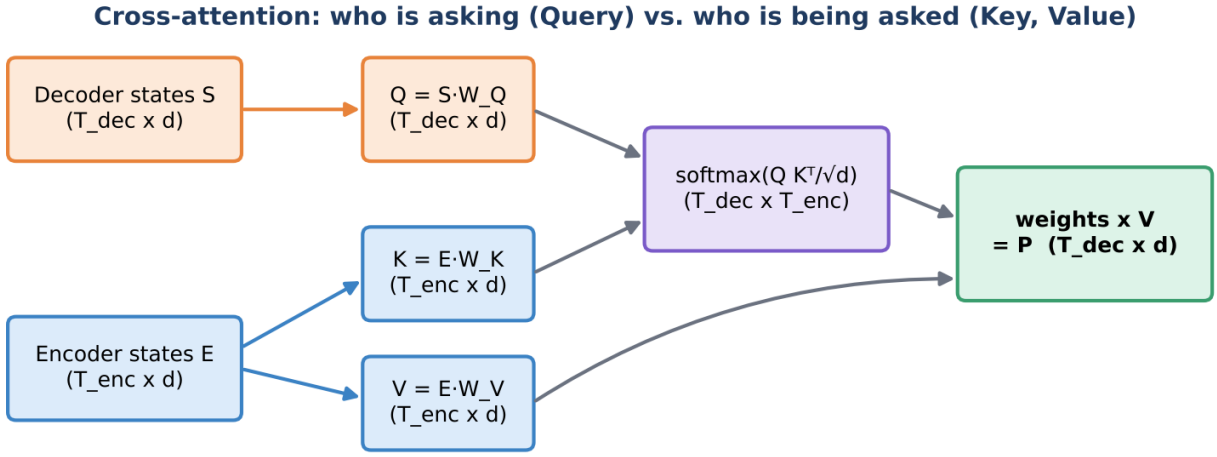


Figure 5. Cross-attention: Q from the decoder, K and V from the encoder. The output has one new vector per decoder position.

$$\mathbf{P = softmax(\ Q_S \cdot K_E^T / \sqrt{d}\ ) \cdot V_E}$$

Because the attention matrix is T_{dec} x T_{enc} and V is T_{enc} x d, the product is T_{dec} x d: still one d-dimensional vector per decoder word. **No mask is needed here** in the lecture's description: the encoder sentence is fully known, so the decoder may look at all of it.

2.3 Step 3: feed-forward network

Each vector goes through a small two-layer network: first a d x d_1 matrix (making it T x d_1), then a d_1 x d matrix (back to T x d). Every position is processed independently.

2.4 Shape tracker (so nothing surprises you)

Stage	Matrix shape	Comment
Decoder input H	T x d	word embeddings (+ positions)
W_Q, W_K, W_V	d x d each	turn d-dim input into d-dim q, k, v
Q, K, V (self-attn)	T x d each	
Attention weights	T x T	lower-triangular after masking
S (after masked self-attn)	T x d	
Cross-attn weights	T_{dec} x T_{enc}	rows = decoder words, columns = encoder words
P (after cross-attn)	T_{dec} x d	
After feed-forward	T x d	d → d_1 → d

2.5 Why masking must be repeated in every layer

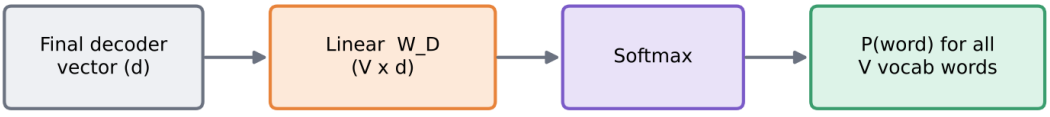
A common question: "We masked in layer 1, so why again in layer 2?" Because information flows **through** vectors.

EXAMPLE

Target words: w1, w2, w3, w4. In layer 1, w4's vector legitimately looked at w1-w3, so it now **contains w3's information**. If, in layer 2, we let w2 look at w4, then w2 indirectly receives w3's information, which is the future for w2. That is a leak. So every decoder layer uses the same mask, and the layers are identical in structure (each with its own weights).

2.6 The output layer

After the last layer each position holds a d-dimensional vector, but we need probabilities over V vocabulary words. We multiply by a matrix **W_D of size V x d** to get V scores, and apply **softmax**. This is done for every position.



d-dim vector → V-dim scores → probabilities; pick argmax as the predicted word

Figure 6. From a d-dimensional vector to a probability for every word in the vocabulary.

2.7 Taking off the training wheels

Always teacher-forcing is like riding a bike with training wheels forever. At inference time the decoder will have **no ground truth**: it must use its own previous outputs. So the lecturer's suggestion: use teacher forcing for the first few epochs while the model is still random, and once it has learned enough, **start feeding its own outputs** to prepare it for the real world.

Quick check	Answer
Where do Q, K and V come from in cross-attention?	Q from the decoder states S; K and V from the encoder output E.
What is the shape of the cross-attention output?	T_{dec} x d (one new d-vector per decoder position).
Why is the mask applied in all decoder layers?	Layer-1 vectors already contain information from earlier words; unmasked later layers would let that leak backwards.
What does W_D do?	Maps a d-dim vector to V scores (V x d matrix), then softmax gives word probabilities.

PART 3 | LECTURE L7

Why Transformers Need Positional Information

Attention treats a sentence like a bag of words. How do we give it an order?

BIG IDEA

Self-attention has **no built-in notion of word order**. We must inject position information into the input embeddings ourselves.

3.1 RNNs had order for free, Transformers do not

In an RNN, h_t depends on h_{t-1} and the current input, so the model processes words one by one and order is naturally baked in. In a Transformer, every word's new vector is computed from **all** words at once:

$$z_1 = \Sigma_{i=1..4} \text{softmax}(q_1 \cdot k_i) \cdot v_i$$

This is a **sum**, and the order in which you add numbers does not matter. Nothing in the formula tells the model which word came first.

EXAMPLE

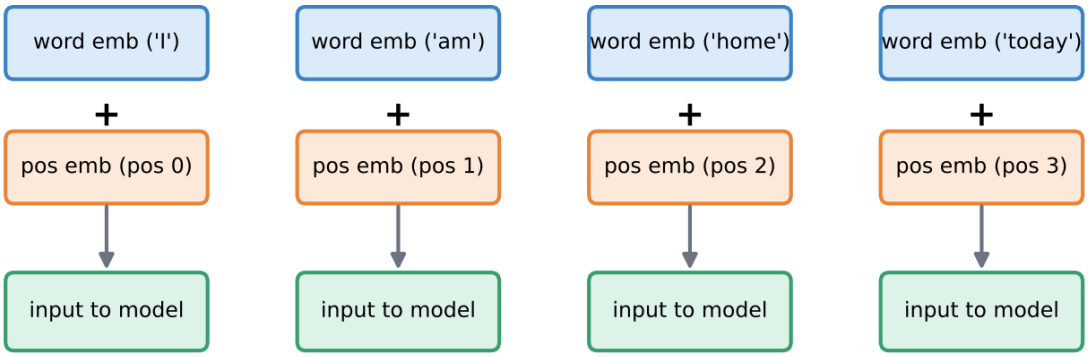
Sentence A: **I am home today**. Sentence B: **I am today home**. Each word's new representation is a weighted sum over the same set of words, so the model cannot tell that "home" is closer to "am" in A. Both sentences look the same to it. This property is called **permutation invariance**.

WATCH OUT

For translation this is a real problem: the encoder passes the same information to the decoder whether the input is "I am home today" or a scrambled version, so it would produce the same translation. Order matters in language, and distance matters too (far-away words are often less relevant).

3.2 Where do we add position information?

At the very input. Each word first becomes a d_{model}-dimensional (e.g. 512) **word embedding**, looked up from a trainable V x 512 embedding matrix. We add a **positional embedding** of the same size, so the sum carries both "what" and "where".



Both vectors have size d_model (e.g. 512), so they can simply be added element-wise.

Figure 7. Input to the model = word embedding + positional embedding (same size, so they can be added).

3.3 What do we want from a positional code?

#	Wish	Why
1	Depends on the position j	Otherwise it carries no position information.
2	Values are on a similar scale as word embeddings (small)	Large numbers would drown out the word information.
3	Works for any sentence length, even longer than seen in training	Test sentences may be longer than training ones.
4	Nearby positions are **close**, far positions are **far**	Lets the model reason about relative distance.

3.4 Naive options and why they fail

Option	Idea	Problem
Constant	Position j → vector of all j's (all 0s, all 1s, all 2s ...)	Position 256 adds 256 to every entry while embeddings are ~0.01-0.05, so the word is drowned. Scaling by 10^{-3} helps the range, but values still grow linearly forever and the scales never match.
One-hot	Position j → vector with a single 1 at index j	Very sparse. Worse, **every pair of positions is exactly √2 apart**, so position 1 is as far from 2 as from 500. No sense of distance.
Learned (random init)	Treat position vectors as trainable parameters	Positions never seen in training (e.g. training max 40, test sentence of length 44) stay random and untrained; rarely seen lengths are barely trained. Also no guarantee that nearby positions end up close.

EXAMPLE

One-hot with 512 positions: position 1 = (0,1,0,0,...), position 3 = (0,0,0,1,...). Distance = $\sqrt{(1^2+1^2)} = \sqrt{2}$. Position 1 vs position 400 is also √2. The code says all positions are equally different.

> **A NOTE ON THE LECTURE'S SCOPE**
>
> This is the **original Transformer** view. Newer models use relative positional encodings, and some recent work even argues positional embeddings may not be needed at all. The lecturer says these will be covered later and to treat this part "with a pinch of salt".

So we want something that is **pre-computed by a formula** (works for any length), has **small bounded values**, depends on j, and makes close positions look similar. That is the next part.

Quick check	Answer
Why is self-attention permutation invariant?	Each output is a sum over all words; the order of adding does not change the sum.
Why add rather than attach the position code?	Both have dimension d_{model}, so they are added element-wise and the model size stays d_{model}.
What is the distance between any two one-hot positions?	$\sqrt{2}$, regardless of how far apart the positions are.
Why can learned positions fail?	Unseen/rare positions are untrained, and nothing forces nearby positions to be similar.

PART 4 | LECTURE L5

Sinusoidal Positional Encoding

A formula that gives every position a unique, distance-aware code

BIG IDEA

Give each position a vector built from **sine and cosine waves of different speeds**. The vector depends on the position, stays within [-1, 1], works for any length, and nearby positions get similar vectors.

4.1 The formula

$$\mathbf{PE(j, i) = \sin(j / 10000^{i / d_model}) \text{ if } i \text{ is even}}$$
$$\mathbf{PE(j, i) = \cos(j / 10000^{i / d_model}) \text{ if } i \text{ is odd}}$$

Symbol	Meaning
j	the **position** of the word in the sentence (0, 1, 2, ... up to the max length T)
i	the **dimension index** inside the vector (0 ... d_{model} - 1); even i use sin, odd i use cos
d_{model}	the size of every vector in the Transformer (e.g. 512); the lecture's toy heat-map uses 64
10000	a constant that sets the slowest wave speed

(Footnote: the original paper pairs sin/cos using $2i/d_{model}$ over index pairs; the lecture uses the simplified indexing shown above. The idea is identical.)

4.2 Hand calculation

EXAMPLE

Position j = 0: sin(0) = 0 for even i, cos(0) = 1 for odd i. So the first row is always **0, 1, 0, 1, 0, 1 ...**
Position j = 4, i = 0: even, so $\sin(4 / 10000^0) = \sin(4) =$ **-0.757** (radians).

Here is a tiny version with $d_{model} = 4$, computed exactly from the formula:

position j	i=0 (sin(j/1))	i=1 (cos(j/10))	i=2 (sin(j/100))	i=3 (cos(j/1000))
0	0.000	1.000	0.000	1.000
1	0.841	0.995	0.010	1.000
2	0.909	0.980	0.020	1.000
3	0.141	0.955	0.030	1.000

Notice that column i = 0 changes quickly with j, while column i = 3 barely changes. That is the next idea.

4.3 Why waves of different speeds?

A bigger denominator stretches the wave out, so it oscillates **more slowly** (lower frequency). The numerator is j (so the value depends on position), and the denominator $10000^{i/d}$ grows with i. So as i increases, the sine/cosine gets slower. Each dimension is a wave with its own speed, and together the 512 values form a unique "fingerprint" for each position.

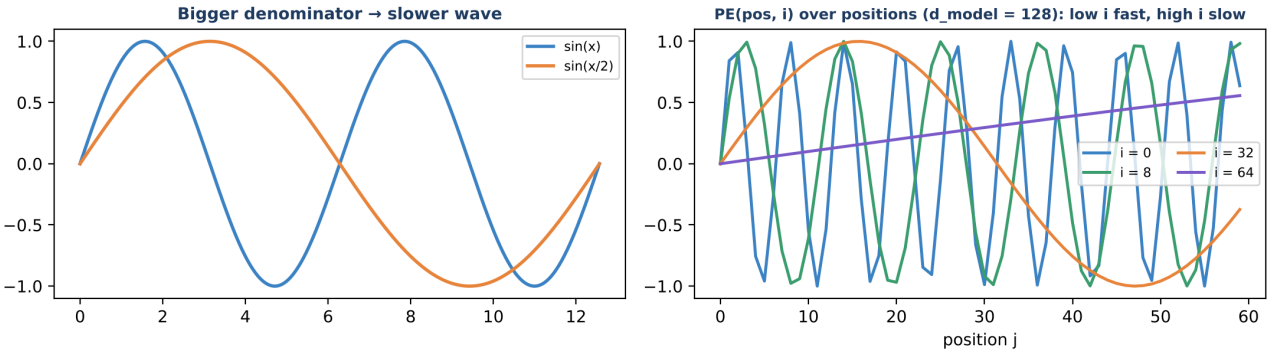


Figure 8. Left: sin(x/2) is a stretched (slower) version of sin(x). Right: low dimension index i = fast wave, high i = slow wave.

EXAMPLE

Analogy (not from the lecture): think of a clock with a seconds hand, a minutes hand and an hours hand. Each hand moves at a different speed, but together they identify the exact time. The fast dimensions tell apart neighbouring positions; the slow dimensions tell far-apart positions apart.

WHY BOTH SIN AND COSINE?

Switching between sin and cos (even/odd i) gives one more axis of variation. Which one is used for even vs odd is arbitrary; we just follow the original paper.

4.4 The heat-map

Each row below is one position's vector, each column is one dimension. Left: 8 positions x 64 dimensions (the lecture's toy example). Right: 96 positions x 512 dimensions. Row 0 is the alternating 0, 1, 0, 1 pattern. Columns on the right side look almost constant because their waves are very slow.

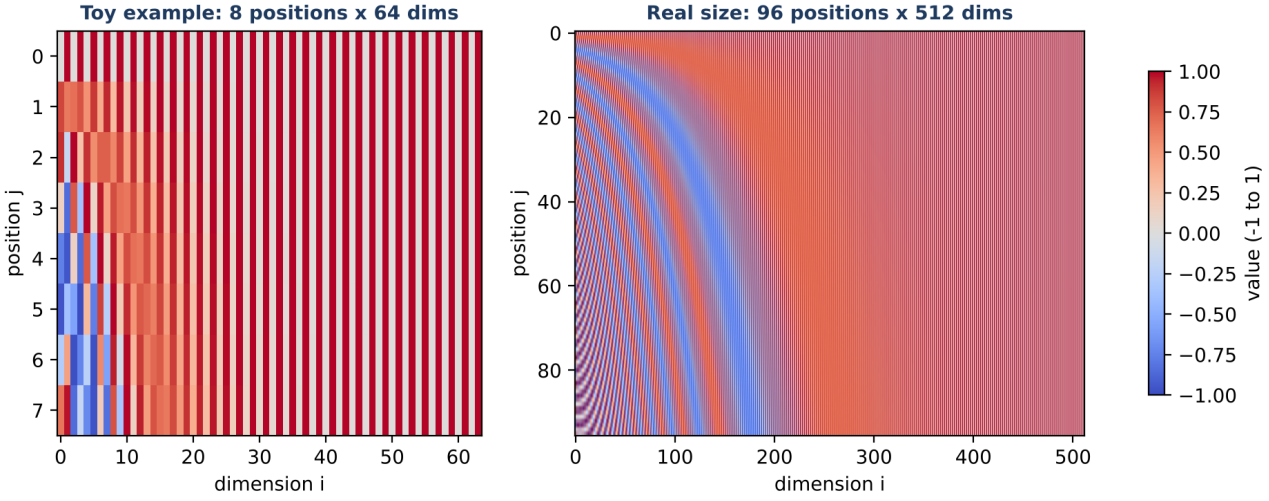


Figure 9. Positional encoding matrices. Close rows look similar; far rows look different.

In the toy heat-map the lecturer notes that after a point the values look like 0, 1, 0, 1: with d_{model} = 64 and i = 32 the denominator is $10000^{32/64}$ = 100, so sin(7/100) ≈ 0.07, nearly zero. The slow dimensions change very little across a short sentence.

4.5 Do close positions really look close?

Yes. Computing the Euclidean distance from one position's code to all others shows exactly the shape we wanted: **zero at itself, growing as you move away**. One-hot codes give a flat line (always √2); random codes show no pattern.



Figure 10. Distance from position 10's code to every other position (our own computation, d_model = 64).

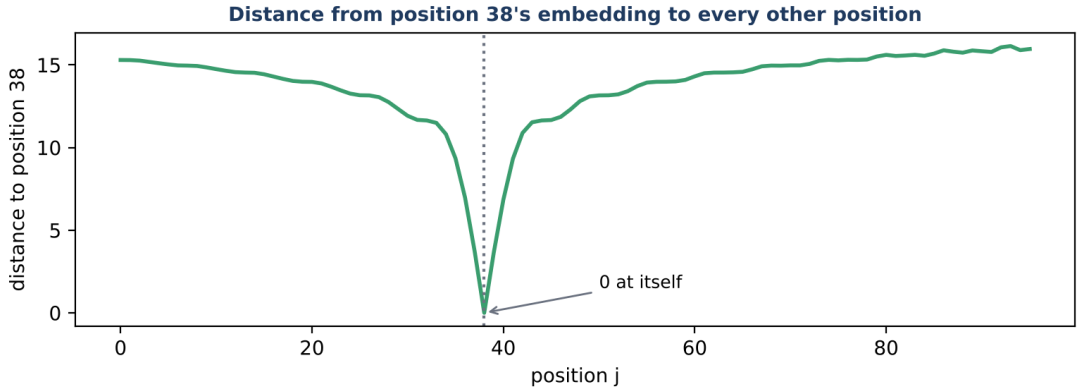


Figure 11. Position 38 in a 96-position, 512-dimension encoding: distance grows smoothly on both sides, like the lecture's animation.

4.6 Scorecard

Wish	Constant	One-hot	Learned	Sinusoidal
Depends on position	yes	yes	yes	**yes**
Small values	no	yes	yes	**yes (-1 to 1)**
Any length (even unseen)	yes	no (fixed size)	no (untrained)	**yes (formula)**
Nearby = close, far = far	no	no (always √2)	not guaranteed	**yes**
Extra parameters to learn	0	0	T x d	**0**

Quick check	Answer
What are j and i in the formula?	j = word position; i = dimension index in the d_{model}-long vector.
What is the encoding of position 0?	0, 1, 0, 1, ... (sin(0) = 0 and cos(0) = 1).
What happens to the wave as i grows?	The denominator grows, so the frequency drops (slower wave).
Can it handle a sentence longer than any in training?	Yes: it is a formula, not a learned table, so any position can be computed.

PART 5 | LECTURE L8

Residual Connections & Batch Normalization

Two tools that make very deep networks trainable

BIG IDEA

Deep stacks are hard to train. **Residual connections** give gradients a shortcut path, and **normalization** keeps each layer's inputs well-behaved so training is stable and fast.

5.1 The depth problem

Each encoder layer has an attention sub-layer plus a feed-forward network with two layers (the lecture counts 3 layers per encoder). Each decoder layer has two attention blocks (self + cross) plus the feed-forward network (counted as 4 layers). With 6 encoders and 6 decoders:

6 x 3 + 6 x 4 = 42 layers

That is deep. Remember ResNet with 152 layers: the worry is **vanishing gradients**. When the loss gradient is back-propagated through many layers it can shrink toward zero, so early layers barely learn.

5.2 Residual (skip) connections

The fix from ResNets: after a transformation F, **also pass the original input around it** and add it:

output = x + F(x)

EXAMPLE

Analogy: a highway with an express lane. The sub-layer F(x) is the winding local road with many traffic lights, while the skip connection is the express lane that carries the original signal (and the gradient) straight through. Even if F learns little at first, the information and gradient still arrive.

REMEMBER

F can be an attention block or a feed-forward block. In the Transformer, **every sub-layer is wrapped like this** (see Part 6).

5.3 Why normalize? The standardization idea

In ML we always **standardize inputs** to zero mean and unit variance: subtract the mean, divide by the standard deviation, computed per feature. But look at a deep network: the output H_1 of layer 1 is the **input** to layer 2, H_2 is the input to layer 3, and so on. Nothing guarantees H_1, H_2, ... are standardized. **Batch normalization** simply applies the same standardization to the inputs of every layer.

Empirically this makes training more stable and convergence faster, especially in deep networks.

5.4 Batch normalization step by step

Notation: $\mathbf{x_i^j}$ = activation of neuron (feature) **i** for training sample **j** in the batch (j = 1..m), at layer **l**. So there are three indices: neuron i, sample j, layer l.

Step	Formula	In words
1. Mean	$\mu_i = (1/m)\ \Sigma_j\ x_i^j$	average of feature i over the m samples of the batch
2. Variance	$\sigma_i^2 = (1/m)\ \Sigma_j\ (x_i^j - \mu_i)^2$	spread of feature i over the batch
3. Standardize	$\hat{x}_i^j = (x_i^j - \mu_i) / \sqrt{(\sigma_i^2 + \varepsilon)}$	zero mean, unit variance; ε is a tiny number to avoid dividing by 0
4. Scale and shift	$\tilde{x}_i^j = \gamma_i \cdot \hat{x}_i^j + \beta_i$	learnable γ and β let the network undo normalization if it wants

EXAMPLE

One feature measured on a batch of 3 samples: **2, 4, 6**. Mean = 4, variance = (4+0+4)/3 = 2.67, std ≈ 1.63. Standardized values: (2-4)/1.63 = **-1.22**, (4-4)/1.63 = **0**, (6-4)/1.63 = **+1.22**. Now the feature has mean 0 and variance 1.

Three questions the lecture answers

- **Can we back-propagate through it?** Yes. The standardization uses only differentiable operations (mean, variance, subtract, divide), so gradients flow. Steps 1-3 introduce **no new parameters**.
- **Why "batch" normalization?** For the input layer we could compute the mean over the whole dataset. But hidden-layer activations are only computed for the current **mini-batch** (e.g. 32, 64, 128 samples), so the statistics must come from that batch.
- **Why γ and β?** Forcing zero mean and unit variance everywhere may hurt if the task wants other values (e.g. all-positive activations). With learnable γ and β the network can recover the original values: if γ = σ and β = μ, then $\tilde{x} = x$. We keep the door open.

5.5 The 3-D picture: batch x layer x feature

Think of all activations as a cube with three axes: **batch** (which sample), **feature** (which neuron), and **layer** (depth). Batch norm takes statistics **along the batch axis** for each feature. Part 6 shows layer norm taking them along a different axis.

Quick check	Answer
What does x + F(x) buy us?	A shortcut path for information and gradients, which fights vanishing gradients in deep stacks.
What does batch norm compute its mean over?	The samples in the current mini-batch, separately for each feature.
Why are γ and β learnable?	So the network can re-scale or even undo the standardization if that helps the task.
Does batch norm add parameters?	Only γ and β (per feature). The standardization itself has none.

PART 6 | LECTURE L10

Layer Normalization & the Full Transformer

Normalizing across features, Add & Norm, and the complete architecture

BIG IDEA

Batch norm struggles in Transformers. **Layer norm** standardizes across the features of a **single token** instead, so it does not depend on batch size. Each sub-layer is wrapped as **LayerNorm(x + Sublayer(x))**.

6.1 Why not just use batch norm?

- **Small batches give poor estimates** of the mean and variance. With batch size 1, the mean is just the value itself and the variance is 0, which is meaningless.
- **Empirically it hurt NLP performance**, and a systematic study confirmed this. (A fix called PowerNorm was published.)
- A simpler and very effective alternative: **layer normalization**.

6.2 Layer normalization

Same abstraction (the batch x layer x feature cube), but we now take the mean and variance **along the feature axis inside one layer's output for one sample**, not along the batch.

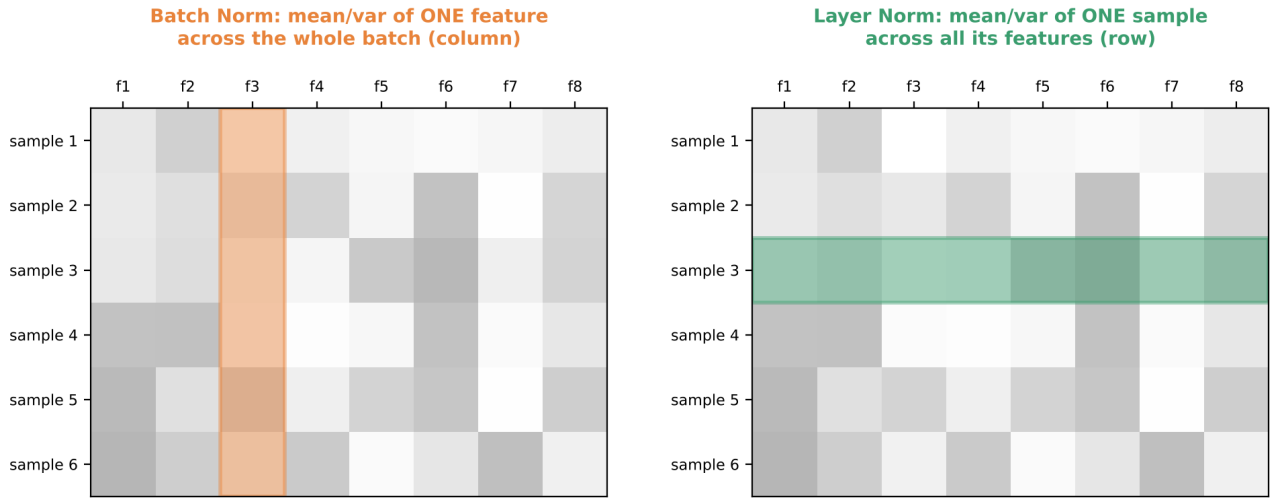


Figure 12. Batch norm averages down a column (one feature across samples); layer norm averages across a row (all features of one sample).

Because d_{model} is large (512, 768, 1024), the mean and variance computed across the features are reliable **even with batch size 1**.

	Batch Norm	Layer Norm
Statistics over	the batch (per feature)	the features of one sample/token
Needs a large batch?	yes	no (works with batch size 1)
Formula	same: standardize, then $\gamma\,\hat{x} + \beta$	same: standardize, then $\gamma\,\hat{x} + \beta$

	Batch Norm	Layer Norm
Used in Transformers	no (hurts NLP)	**yes**

$$\mu = (1/H)\ \Sigma_k\ h_k,\ \sigma^2 = (1/H)\ \Sigma_k\ (h_k - \mu)^2,\ \tilde{h}_k = \gamma \cdot (h_k - \mu)/\sqrt{(\sigma^2 + \varepsilon)} + \beta$$

Here H is the number of neurons (features) in the layer, so the sums run over the vector's own entries.

EXAMPLE

One token's vector with 4 features: **[1, 3, 5, 7]**. Mean = 4, variance = (9+1+1+9)/4 = 5, std ≈ 2.24. Normalized: [-1.34, -0.45, +0.45, +1.34]. Note that we used **only this one token**; it does not matter what other sentences are in the batch.

6.3 Add & Norm: residual + layer norm together

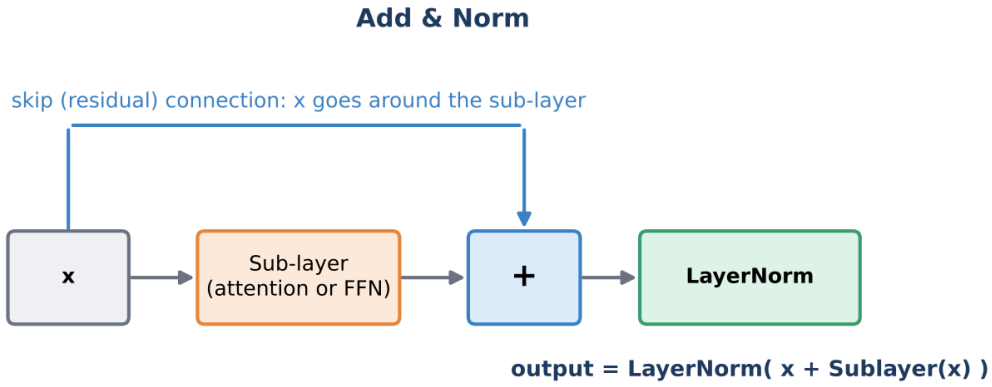


Figure 13. The Add & Norm block placed after every sub-layer.

Take the encoder's attention sub-layer. Let **x** be its input and $\hat{\mathbf{x}}$ the attention output:

$$\tilde{x} = \text{LayerNorm}(\ x + \hat{x}\ )$$

Then $\tilde{x}$ goes into the feed-forward network, producing a new output, and again:

$$x' = \text{LayerNorm}(\ \tilde{x} + \text{FFN}(\tilde{x})\ )$$

So after **every attention layer** and **every feed-forward network** we do Add & Norm: add the input back (residual), then normalize.

6.4 The complete Transformer

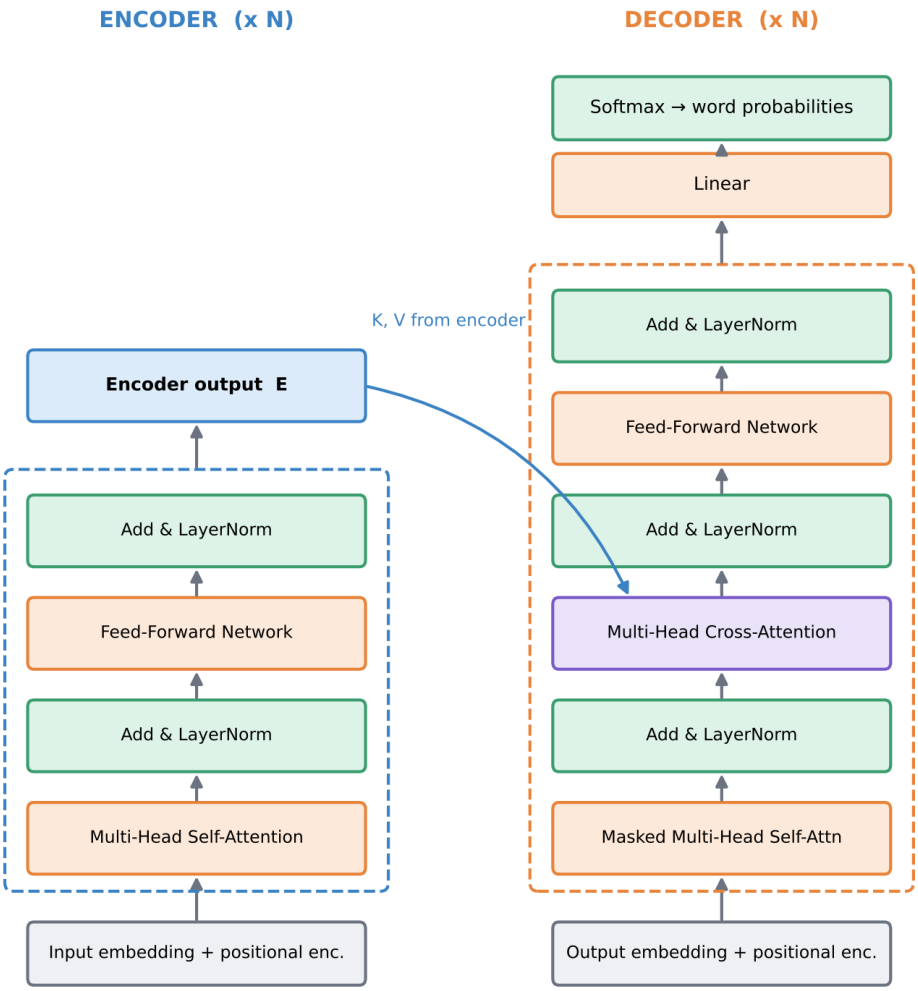


Figure 14. The full architecture (N layers of each stack, e.g. N = 6).

Encoder, one layer (repeated N times)

- Word embedding **+ sinusoidal positional encoding** (each 512-d, added).
- From each vector compute q, k, v with the W matrices → **multi-head self-attention**.
- **Add & LayerNorm**.
- **Feed-forward network → Add & LayerNorm**.
- Output E goes to every decoder layer's cross-attention.

Decoder, one layer (repeated N times)

- Output embedding (e.g. embedding of the word just generated; first input is <GO>) **+ positional encoding**.
- **Masked** multi-head self-attention → Add & LayerNorm.
- **Cross-attention**: Q from the decoder, K and V from the encoder → Add & LayerNorm.
- Feed-forward → Add & LayerNorm.
- After the last layer: **Linear** (d → V) and **Softmax** to get a word distribution.

6.5 Every component and its job

Component	Why it is there
Multi-head self-attention	each word gathers information from other words (softmax($Q^T K/\sqrt{d}$)V)
Positional encoding	gives the model a sense of word order
Masked self-attention	teacher forcing without leaking the future; also hides PAD tokens
Cross-attention	decoder looks at the encoded input sentence
Feed-forward network	per-position non-linear processing
Residual connection	gradients and information flow through deep stacks
Layer normalization	stabilizes and speeds up training, independent of batch size
Linear + softmax	converts final vectors into word probabilities

Quick check	Answer
Why is layer norm preferred over batch norm in Transformers?	It does not depend on batch size and works better for NLP; statistics come from the d_{model} features of one token.
What does Add & Norm compute?	LayerNorm(x + Sublayer(x)).
How many Add & Norm blocks in one encoder layer? In one decoder layer?	Encoder: 2 (after attention, after FFN). Decoder: 3 (after masked self-attn, cross-attn and FFN).
What is the very last step of the decoder?	Linear layer then softmax over the vocabulary.

CHEAT SHEET

Formulas & Shapes on One Page

Review this the night before the exam

Topic	Key formula / fact
Autoregressive decoding	predict next word from encoder output + words generated so far; output = softmax over vocabulary (V-dim)
Teacher forcing	train with the **true** previous word as input, but loss still on the prediction; later switch to own outputs
Masked attention	softmax($Q^T K/\sqrt{d} + M$) V; M = 0 on/below diagonal, -∞ above; fixed T x T matrix
Padding mask	-∞ on PAD positions so real words do not attend to them
Cross-attention	Q from decoder (S), K and V from encoder (E); output T_{dec} x d
Shapes	H, Q, K, V, S, P: T x d; $W_{Q,K,V}$: d x d; attention: T x T; FFN: d → d_1 → d; W_D: V x d
Why mask at every layer	later layers would otherwise let a word read future information stored in other words' vectors
Permutation invariance	attention is a sum over words, so order is lost without positional info
Bad position codes	constant (scale grows), one-hot (all pairs √2 apart), learned/random (unseen lengths, no distance structure)
Sinusoidal PE	PE(j,i) = $\sin(j / 10000^{i/d})$ for even i, cos(...) for odd i; PE(0) = 0,1,0,1,...; added to word embedding
Residual	output = x + F(x)
Batch norm	per feature over the batch: $\hat{x} = (x-\mu)/\sqrt{(\sigma^2+\varepsilon)}$; $\tilde{x} = \gamma\hat{x} + \beta$
Layer norm	same formula but μ, σ^2 over the features of one token; independent of batch size
Add & Norm	LayerNorm(x + Sublayer(x)) after every attention and FFN block
Output	Linear (V x d) then softmax, take argmax (or other decoding strategy)